MULTICORE: classical and post-quantum halves on separate CPU cores (feasibility test)

Date: 2026-11-11. Board: VisionFive 2 (JH7110, **4x SiFive U74 cores**, up to 1.5 GHz), OpenSSL 3.5.9. Compared with the **BASELINE** way of working: everything one after another on a single core.

The idea

A hybrid handshake does every cryptographic step twice, once classically and once post-quantum, and the two halves don't depend on each other. The board has 4 cores, but OpenSSL and our client use only one. If the two halves run on two cores at the same time, the step takes as long as the slower half instead of the sum of both.

The client-side work of one hybrid handshake consists of three such pairs:

Pair	Classical half	Post-quantum half	When it happens in the handshake
`auth`	ECDSA P-256 signature verify	ML-DSA-44 signature verify	server authentication
`kex_keygen`	X25519 key generation	ML-KEM-768 key generation	before the ClientHello
`kex_decaps`	X25519 shared secret	ML-KEM-768 decapsulation	after the ServerHello

Method

Test program `parverify` (`tools/parverify.c`), linked against the OpenSSL builds on the board. Each pair is run in three ways:

Mode	How
`seq`	both halves one after another on one core (today's behaviour)
`par_create`	classical half on a **new thread per operation** (`pthread_create + join`)
`par_pool`	classical half on a **persistent worker thread** created once and woken per operation (how a real implementation would do it)

- 200 iterations per pair and mode; **the three modes rotate every iteration**, so they see the same conditions.
- 100 ms pause before every measurement, so the CPU idles in between like between real handshakes (this matters for the power-saving clock settings).
- Run at three CPU clock settings (`default` = ondemand, min 312.5 MHz, the baseline; `min750` = ondemand, min 750 MHz; `fixed1500` = performance, 1.5 GHz) and with two OpenSSL builds (`opt` = plain rv64gc, `bitmanip` = Zba/Zbb).
- Every operation is self-checked before timing (signatures verify, keys/secrets are produced); **0 failures** in 10,800 measurements. The core each half ran on was recorded (`sched_getcpu`).
- The CPU setting was restored to the default automatically at the end.

Files: `multicore_feasibility_raw.csv` (`label,pair,mode,iter,us,cpu_classical,cpu_pq`), `multicore_feasibility_stats.txt`, `multicore_feasibility.log`, `scripts/parverify_run.sh`.

Results

Authentication pair (ECDSA verify + ML-DSA-44 verify), median time for both halves

Build	Clock	One core (seq)	Two cores, new thread	Two cores, worker thread	Saved	Speed-up	Parallel faster in
plain	default	9.80 ms	6.19 ms	**5.97 ms**	**–3.83 ms**	1.64x	198/200
plain	min 750 MHz	4.89 ms	3.09 ms	**2.95 ms**	–1.94 ms	1.66x	185/200
plain	1.5 GHz	2.44 ms	1.55 ms	**1.47 ms**	–0.96 ms	1.65x	200/200
bitmanip	default	9.26 ms	6.25 ms	**6.03 ms**	–3.23 ms	1.54x	189/200
bitmanip	min 750 MHz	4.62 ms	3.11 ms	**2.98 ms**	–1.64 ms	1.55x	192/200
bitmanip	1.5 GHz	2.30 ms	1.56 ms	**1.49 ms**	–0.81 ms	1.54x	190/200

Key exchange pairs, median time for both halves

Build	Clock	Keygen: seq → worker	Saved	Decaps: seq → worker	Saved
plain	default	2.90 → **1.91 ms**	–0.99 ms	3.31 → **2.21 ms**	–1.10 ms
plain	min 750 MHz	1.45 → **0.96 ms**	–0.49 ms	1.65 → **1.10 ms**	–0.55 ms
plain	1.5 GHz	0.73 → **0.48 ms**	–0.25 ms	0.83 → **0.55 ms**	–0.27 ms
bitmanip	default	2.55 → **1.56 ms**	–0.99 ms	3.00 → **1.91 ms**	–1.10 ms
bitmanip	min 750 MHz	1.27 → **0.78 ms**	–0.49 ms	1.50 → **0.95 ms**	–0.55 ms
bitmanip	1.5 GHz	0.64 → **0.39 ms**	–0.25 ms	0.75 → **0.48 ms**	–0.27 ms

All client-side crypto of one hybrid handshake (sum of the three pairs, medians)

Build	Clock	One core	Two cores (worker thread)	Saved	Speed-up
plain	**default (baseline setting)**	16.01 ms	**10.09 ms**	**–5.92 ms**	1.59x
plain	min 750 MHz	7.99 ms	5.01 ms	–2.98 ms	1.59x
plain	1.5 GHz	3.99 ms	2.51 ms	–1.48 ms	1.59x
bitmanip	default	14.81 ms	9.49 ms	–5.32 ms	1.56x
bitmanip	min 750 MHz	7.39 ms	**4.71 ms**	–2.68 ms	1.57x
bitmanip	1.5 GHz	3.69 ms	2.36 ms	–1.33 ms	1.56x

Were two cores really used?

`par_create`: 99–100 % of runs on two different cores. `par_pool`: 91–100 %. In the remaining runs the Linux scheduler put the worker on the same core as the main thread; pinning the threads to fixed cores (`pthread_setaffinity_np`) would remove that and the occasional slow outlier (the higher p95 of `par_pool`).

Findings

1. **The approach works: 1.5–1.65x faster for every pair, at every clock, with both builds.** The parallel version won in 90–100 % of individual iterations. The speed-up stays below 2x because the two halves aren't equal in length (for example ECDSA verify 1.41 ms vs ML-DSA-44 verify 0.84 ms at 1.5 GHz): two cores can only shorten the step to the slower half. For authentication the ideal is about 1.7x, and the worker-thread version reaches 1.65x.
2. **The saving is largest exactly where the board is slowest.** At the default power-saving clock the client crypto drops from 16.0 to 10.1 ms (–5.9 ms). At 1.5 GHz the saving is –1.5 ms. Multicore therefore helps

most in the low-heat configuration.
3. **A persistent worker thread beats creating a thread per operation** (about 0.1–0.2 ms per pair at the default clock): thread creation is a real cost on this CPU at low clock.
4. **The approaches stack:** bitmanip + min 750 MHz + two cores brings the client crypto to **4.71 ms**, versus **16.01 ms** for the baseline setup (plain build, default clock, one core): −11.3 ms, 3.4x less.
5. **Correction of my earlier estimate:** before measuring I estimated the authentication saving at about 0.84 ms at full clock and 2–3 ms at the default clock. Measured: 0.96 ms and 3.8 ms, so the effect is larger than estimated.

What this does NOT show yet (honest limits)

- **This is a feasibility test of the crypto steps, not an end-to-end TLS measurement.** The savings won't translate 1:1 into connection time until the TLS stack is changed:
- **Key exchange:** X25519 and ML-KEM run inside OpenSSL's built-in X25519MLKEM768 implementation, one after the other. Running them in parallel means modifying OpenSSL's hybrid KEM code (`providers/implementations/kem/`).
- **Authentication:** in our current design the ML-DSA signature arrives after the handshake, in the HTTP response, while ECDSA is verified during the handshake, so the two never overlap. To overlap them, the server must send the ML-DSA signature with the certificate (for example as a TLS 1.3 certificate extension, bound to the handshake randoms and key share) so the client can verify both at the same time.
- **Projection (estimate, not measured):** the baseline hybrid connection on the LAN at the default clock takes 27.95 ms. If the full −5.9 ms crypto saving carried over, it would be about 22 ms, close to the legacy baseline of 21.6 ms, at the default clock with no extra heat. This needs the implementation above to confirm.
- **Energy** wasn't measured: the same amount of computation is done, but on two cores for a shorter time.
- Only one connection at a time was tested. With many simultaneous connections all cores are busy anyway, and the benefit shrinks; multicore helps the latency of a single connection, not throughput.

Suggested next steps

1. **Parallel authentication in the real handshake:** send the ML-DSA signature in a TLS 1.3 certificate extension (`SSL_CTX_add_custom_ext`, context `SSL_EXT_TLS1_3_CERTIFICATE`), start the ML-DSA verification on a worker thread as soon as the certificate arrives, let OpenSSL verify ECDSA in the meantime, join before accepting. Then measure end-to-end with 1000 runs, like the other approaches.
2. Pin the worker thread to a dedicated core.
3. Parallel key exchange inside OpenSSL's hybrid KEM, as a patch to OpenSSL.

Source: multicore_REPORT.md